

SEQ-REFACTOR: A Graph-Theoretic Formalisation of Smell-Dependency Ordering for Safe Multi-Smell Agentic Refactoring of Large-Scale Software Systems

Munura Maihankali and Ivan Polášek

*Department of Computer Science, Faculty of Mathematics, Physics and Informatics
Comenius University Bratislava, Slovakia
maihankali1@uniba.sk*

Abstract. Large language model agents can now perform individual behaviour-preserving refactorings with accuracy that approaches human developers, yet they remain unreliable on realistic modules that carry many interacting code smells at once. A central and largely unexamined reason is that the *order* in which co-occurring smells are resolved is a correctness concern rather than an implementation detail: resolving a God Class relocates the methods that later determine which Feature Envy instances exist, and addressing Shotgun Surgery before Divergent Change drives the module through structurally inconsistent intermediate states. Current automated and agentic refactoring tools operate at the level of a single smell in a single file and treat smell instances independently, so they cannot reason about these prerequisites. This paper introduces SEQ-REFACTOR (SEQ for the *sequencing* of multi-smell refactorings), a formal, graph-theoretic treatment of smell-dependency ordering framed specifically as a behavioural-safety property for agentic refactoring. Prior search-based work schedules refactorings to optimise quality or effort; we instead treat dependency-correct ordering as a safety constraint on the intermediate states of an agentic pipeline. We model the detected smells of a module as a smell-dependency graph whose directed edges encode structural prerequisites, we define an impact score that combines coupling, complexity, and smell co-occurrence, and we cast safe execution as the search for a linear extension of the induced partial order that brings high-impact smells forward. We give an algorithm that unifies impact-prioritised best-first selection with dependency-respecting topological ordering through a priority-queue formulation of **Kahn's algorithm**, and that surfaces dependency cycles as escalation tasks by strongly-connected-component decomposition, all in $O((|V| + |E|) \log |V|)$ time. The ordering drives a stateful LangGraph pipeline that grounds generation with structural retrieval and gates every transformation on a five-family metric and test verification with rollback. We specify a controlled evaluation against unordered, impact-only, and topology-only baselines on synthetic Java subjects with known dependency structure and on open-source modules, and we report a directional pilot on an eight-smell synthetic module that illustrates the cascading violations that dependency-aware ordering avoids.

Index Terms. Software refactoring, code smells, agentic artificial intelligence, large language models, topological sort, technical debt, program analysis, software maintainability, code quality.

I. Introduction

Large-scale software systems decay. As requirements evolve and delivery pressure accumulates, structure drifts away from intent, coupling grows, and local shortcuts compound into technical debt whose interest is paid as additional effort on every subsequent change [10], [11], [4]. Refactoring, the disciplined restructuring of code without changing its external behaviour [4], [5], is the principled remedy: it improves maintainability incrementally and verifiably, and it preserves the domain knowledge that a rewrite would discard.

Automated support for refactoring has advanced quickly. Rule-based tools detect and apply individual transformations deterministically, and history-based detectors recover smells from version histories [12], [13], [22], and a recent wave of LLM-based and agentic systems generate refactorings with contextual reasoning that rule catalogues cannot match. Method-level tools suggest and apply Extract Method or Move Method with safety checks [14], [15], and multi-agent systems that pair generation with retrieval and review now achieve compilation and test-pass rates comparable to human refactorings on class-level tasks [16], [17]. These results are genuine, but they share a boundary: they operate on a single smell instance, usually within a single class or file.

Realistic modules are not like that. A decaying class typically exhibits several smells at once, and those smells interact. Empirical study of inter-smell relations shows that co-located smells interact through shared dependencies and jointly worsen maintainability, and that these interactions are associated with greater maintenance difficulty rather than being independent defects [21], [31]. The practical consequence is that the smells of a module cannot be resolved in arbitrary order. Resolving a God Class by extracting collaborators changes which methods are envious of which data, so the set of Feature Envy instances that remain is defined only after the God Class is addressed. Addressing Shotgun Surgery before the Divergent Change that induces it produces intermediate states in which the same conceptual change is still scattered, wasting effort and, worse, presenting the generator with an inconsistent structure to reason over. Order, in short, is a safety property of a multi-smell refactoring plan, not an implementation detail.

The evidence that this is the operative constraint is now strong. RefactorBench, a benchmark of one hundred handcrafted multi-file refactoring tasks, reports that a capable baseline agent solves only 22% of tasks under base instructions while a human developer under a short time limit solves 87%, and that conditioning the agent on an explicit representation of accumulated state improves task completion by roughly 44% [18]. Multi-file, stateful reasoning across interacting changes is precisely where autonomous refactoring fails, and precisely what a principled ordering discipline addresses.

Prior work has approached related questions. Search-based software engineering formulates refactoring scheduling and sequencing as an optimisation problem, ordering or selecting refactorings under conflict and dependency constraints to maximise quality gain or minimise effort [29], [30], and refactoring tools have long reasoned about the pre- and post-conditions under which one transformation enables or invalidates another. A separate line of work prioritises which smells to fix by impact or severity [20]. To the best of our knowledge, however, this prior work optimises for quality or effort rather than treating dependency-correct ordering as a behavioural-safety property, and it predates the agentic setting in which an LLM generator, lacking the full dependency context, silently produces cascading violations. Impact prioritisation additionally answers a different question (which smell is most valuable to fix) than the one that governs safety (which smells must be fixed before which others). Our contribution is the specific combination of a safety-framed smell-dependency formalisation, an impact-forward yet dependency-correct ordering algorithm, and a stateful agentic pipeline that re-detects as it proceeds; we position it against this prior work rather than claiming ordering itself is unstudied.

A. Contributions

C1: A formal model of smell-dependency ordering. We define the smell-dependency graph, give precise semantics for its prerequisite edges together with a principled derivation of those edges, define a normalised impact score over coupling, complexity, and co-occurrence, and cast safe multi-smell refactoring as the search for an impact-forward linear extension of the induced partial order (Section IV).

C2: An algorithm that unifies priority and safety, with cycle escalation. We present an algorithm that merges impact-prioritised best-first selection with dependency-respecting topological ordering through a priority-queue formulation of Kahn's algorithm, and that decomposes dependency cycles into strongly-connected components surfaced as human-escalation tasks rather than resolved blindly. We analyse its correctness and its $O((|V| + |E|) \log |V|)$ complexity (Section V).

C3: A stateful agentic execution architecture. We specify a LangGraph-orchestrated pipeline that executes the ordered agenda: one smell at a time, grounds generation with vector and structural retrieval to reduce context blindness, gates each transformation on a five-family metric and test verification with rollback, and re-detects after every accepted step so that the agenda stays correct as structure changes (Section VI).

C4: A controlled experimental design. We define research questions, hypotheses, baselines (unordered, impact-only, topology-only), ground-truth synthetic subjects with known dependency structure, open-source subjects, and dependent measures

including cascading-violation rate, net smell resolution, ordering validity, and escalation rate (Section VII).

C5: A directional pilot. We report an illustrative pilot on an eight-smell synthetic Java module that demonstrates feasibility of the pipeline and the specific cascading failure that dependency-aware ordering prevents, framed honestly as a single directional case rather than a statistical result (Section VIII).

B. Scope

The object of study is the *ordering* of behaviour-preserving structural refactorings across multiple co-occurring smells. We do not propose a new smell detector, and we do not propose a new decision procedure for behavioural equivalence; both are treated as components with clearly specified interfaces. The current instrument targets Java, which admits mature static analysis and a rich body of ground-truth refactorings; generalisation to dynamically typed languages is future work. Throughout, we report the pilot as measured and we do not claim aggregate improvement rates in advance of the full study specified in Section VII.

II. Background

A. Code Quality and Code Health

Code quality describes the current state of a codebase along dimensions such as correctness, maintainability, reliability, and efficiency. Code health describes its trajectory over time: whether quality properties are stable or deteriorating across the version history, and whether the structure continues to admit safe incremental change [11]. Refactoring targets both. It improves the present state and, by keeping abstractions aligned with intent, it slows future decay. The dimension most directly addressed by refactoring is maintainability, because refactoring restructures internal form while leaving external behaviour fixed.

B. Code Smells, Architectural Smells, and Anti-Patterns

Code smells are surface indicators in source code that suggest deeper structural problems [4]. Familiar instances include God Class (a single class that accumulates disproportionate responsibility), Long Method, Feature Envy (a method more interested in another class's data than its own), Duplicated Code, Message Chains, and large conditional dispatch. Architectural smells operate at the level of modules and layers: cyclic dependencies that prevent independent deployment, layer violations, and erosion of the intended design. Anti-patterns such as the Blob and Spaghetti Code describe recurring poor solutions at design scale. Smells are the triggers for

refactoring: they localise where restructuring is warranted, and they are the units over which an ordering must be computed.

C. Refactoring as a Behaviour-Preserving Transformation

Refactoring alters internal structure while preserving observable behaviour along every path: input-output relationships, public interfaces, effects on external systems and data stores, and business rules are all held fixed [4], [5]. This distinguishes refactoring from bug fixing, which changes behaviour by design, from optimisation, which alters the non-functional execution profile, and from feature work, which expands the external contract. The distinction matters for this paper because a multi-smell plan must preserve behaviour at every intermediate step, not merely at its end, which is one reason order cannot be arbitrary.

D. Structural Metrics

Structural quality is quantified by a small number of metric families that are computed before and after each transformation and that jointly summarise a unit's health. Cohesion is captured by Lack of Cohesion of Methods and its variants [2]. Coupling is captured by Coupling Between Objects and by afferent and efferent coupling, from which Martin's instability index and distance from the main sequence follow [2], [9]. Complexity is captured by McCabe's cyclomatic complexity [1], by Halstead's measures, and by the composite Maintainability Index [3]. Readability is captured by learned and feature-based models [6], [7], and understandability by Cognitive Complexity [8]. Architectural health is captured by distance from the main sequence and by interface surface. These five families (cohesion, coupling, complexity, readability, and architecture) constitute the verification signal used by the gate in Section VI; the contribution of the present paper is orthogonal to the specific metrics chosen.

E. LLMs and Agentic Refactoring

LLMs bring code reasoning, smell recognition beyond fixed catalogues, and multi-step planning to refactoring, and agentic frameworks wrap these abilities in tool use, memory, and iterative validation. Their well-documented failure mode is hallucination: plausible transformations that compile and pass shallow checks yet violate invariants, generated without awareness of dependency context that lies outside the model's window [18], [16]. The response adopted here follows the emerging consensus that verification must be externalised from the model into the pipeline architecture, and it adds the observation that *planning* must likewise be externalised: the order of operations is a structural property of the module that a generator should be given, not asked to infer.

III. Related Work

A. Rule-Based and IDE Refactoring

Deterministic tools detect specific smells and apply catalogued transformations with strong safety guarantees, and refactoring-mining tools recover applied refactorings from version history [12], [13]. These systems are reliable within their templates but treat each smell instance in isolation and offer no cross-smell planning; the developer decides what to do first.

B. LLM-Based and Multi-Agent Refactoring

EM-Assist and iSMELL couple LLM generation with static tooling to perform method-level refactorings safely [14], [15]. MANTRA combines multiple agents with retrieval-augmented generation to reach human-comparable compilation and test-pass rates, and RefAgent uses specialised agents to reduce smells substantially across Java projects [16], [17]. These advances are important, but they operate at the granularity of a single smell within a single class, and none constructs or reasons over a dependency structure across the smells of a module.

C. The Multi-File and Multi-Smell Constraint

Benchmarks that raise the granularity to multiple files expose a sharp capability gap. RefactorBench reports the 22% versus 87% gap between agents and humans and shows that state conditioning is the largest single lever [18]. Benchmarks of architectural and fine-grained refactoring reach the same conclusion from different angles [19]. This body of evidence motivates a discipline that makes cross-smell dependencies explicit and executable.

D. Smell Prioritisation and Co-Occurrence

A prioritisation literature ranks smells by severity or impact to focus limited refactoring effort, for example the SpIRIT approach to prioritising smelly classes [20]. This answers which smells to fix first for value, but it does not model which smells are prerequisites of which. A parallel literature establishes that smells co-occur and interact, that these interactions propagate through dependencies, and that interacting smells should be prioritised over isolated ones [21], [31]. This work supplies strong empirical grounding for the existence of prerequisite relations between smells, yet stops short of an ordering algorithm.

E. Search-Based Refactoring Scheduling

The closest body of prior work is search-based software engineering applied to refactoring. This literature formulates the selection and ordering of refactorings as an

optimisation problem, using evolutionary and other metaheuristic search to produce refactoring sequences that improve quality metrics or reduce effort, and several approaches model conflicts and dependencies between candidate refactorings so that an inapplicable or redundant transformation is not scheduled [29], [30]. Classical work on refactoring tools similarly reasons about the pre- and post-conditions under which one refactoring enables or invalidates another. This work is the natural point of comparison for ours, and it establishes that ordering refactorings under constraints is a studied problem. It differs from SEQ-REFACTOR in three respects that we make explicit rather than obscure. First, its objective is quality or effort optimisation, whereas we treat dependency-correct ordering as a behavioural-safety constraint on every intermediate state. Second, it schedules transformations chosen by the search itself, whereas we order the smells of a module and drive an agentic generator whose failure mode, silent behavioural divergence under missing dependency context, does not arise for deterministic transformation operators. Third, it predates and does not address the stateful, re-detecting agentic pipeline in which our ordering operates. We therefore position our contribution as a safety- and agent-oriented complement to scheduling, not as the first treatment of refactoring order.

F. The Gap

Taken together, the prior work establishes that smells interact, that order matters, and that refactoring sequences can be optimised for quality; what remains open, to the best of our knowledge, is a treatment that unifies a formal smell-dependency graph, impact-prioritised selection, dependency-correct topological execution, and principled escalation of dependency cycles, framed as a behavioural-safety property and embedded in a stateful agentic pipeline that re-detects as it proceeds. SEQ-REFACTOR targets this combination. We state this as a positioning against the literature above rather than as a claim that no related problem has been studied, and we return in Section IX to the threats this framing carries.

IV. Problem Formalisation

A. The Smell-Dependency Graph

Let a module under refactoring contain a set of detected smell instances. We represent them as the vertices of a directed graph.

Definition 1 (Smell-dependency graph). *The smell-dependency graph of a module is a directed graph $G = (V, E)$ in which each vertex $v \in V$ is a detected smell instance, typed by its smell category and localised to a set of code elements $loc(v)$ (methods, fields, or classes), and each directed edge $(u, v) \in E$ asserts that u is a structural prerequisite of v : resolving v before u is either unsafe, in that it cannot preserve behaviour at the intermediate step, or unstable, in that it drives the module*

through a structurally inconsistent state that a prerequisite-respecting order would avoid.

An edge (u, v) therefore constrains execution: u must be resolved no later than v . The relation $<$ defined by the transitive closure of E is a strict partial order whenever G is acyclic. When G contains cycles, no linear order can respect all constraints simultaneously, a case handled explicitly in Section V.

B. Deriving the Edges

The edge set is derived from two complementary sources, and every edge records its provenance so that an ordering can be audited.

Catalogue rules. A curated rule set grounded in the refactoring literature encodes precedence relations between smell categories [4]. Representative rules include: resolve God Class and Large Class before Feature Envy, because extracting collaborators from an over-large class relocates the very methods whose data affinity defines the remaining envy; resolve Duplicated Code before Long Method, because consolidating duplication changes the boundaries along which a long method should be decomposed; and resolve Divergent Change before Shotgun Surgery when they share elements, because separating unrelated responsibilities in one place removes the scatter that the other would otherwise chase. Table I lists an illustrative subset.

Structural co-location. Beyond category-level rules, an edge is added between two instances when they share code elements and the resolution of one is known to modify elements on which the other structurally depends. This grounds the catalogue rules in the concrete elements of the module and captures cases the categorical rules miss. It is directly supported by the empirical finding that interacting, co-located smells propagate through shared dependencies [21], [31].

TABLE I. Illustrative subset of catalogue precedence rules. An entry “ $A \Rightarrow B$ ” means an instance of A is a structural prerequisite of a co-located instance of B .

Prerequisite A	Dependent B	Rationale
God Class	Feature Envy	extraction relocates envious methods
Large Class	Long Method	class split re-scopes method boundaries
Duplicated Code	Long Method	consolidation changes decomposition seams
Divergent Change	Shotgun Surgery	separating concerns removes scatter
Message Chains	Middle Man	shortening chains exposes idle delegation

C. The Impact Score

Among the many valid execution orders, some deliver quality improvement earlier than others. To prefer high-value smells we attach to each vertex a normalised impact score.

$$Impact(v) = \alpha \hat{c}(v) + \beta \hat{x}(v) + \gamma \hat{f}(v) \quad (1)$$

where the three terms are, respectively, the coupling contribution, the complexity contribution, and the co-occurrence frequency of the smell instance, each normalised to $[0, 1]$ across the module, and $\alpha, \beta, \gamma \geq 0$ with $\alpha + \beta + \gamma = 1$ are weights whose sensitivity is examined in Section VII. The coupling and complexity contributions are read from the metric families of Section II-D for the elements in $\text{loc}(v)$; the co-occurrence term rewards smells that participate in many interactions, consistent with the evidence that interacting smells carry disproportionate maintenance cost [21].

D. The Ordering Problem

Let $L(G)$ denote the set of linear extensions of G , that is, the permutations π of V that respect every edge: if $(u, v) \in E$ then u precedes v in π . Respecting E is a hard safety constraint. Among the safe orders we prefer those that bring impact forward, which we formalise with a discounted cumulative objective.

$$J(\pi) = \sum_{i=1}^{|V|} \delta^{i-1} \text{Impact}(\pi_i), \quad \delta \in (0, 1] \quad (2)$$

so that a smell placed earlier in π contributes more, and the discount δ controls how strongly early impact is favoured. The ordering problem is then

$$\pi^\star = \arg \max_{\pi \in L(G)} J(\pi) \quad (3)$$

We do not claim to solve (3) optimally; the priority-driven algorithm of Section V is a greedy approximation whose behaviour and quality we evaluate empirically. Framing the goal as (3) makes explicit both the hard constraint (membership in $L(G)$) and the soft objective (impact-forward placement), and it separates the two so that safety is never traded for priority.

E. Why Order Matters: A Worked Example

Consider a class exhibiting, among others, a God Class instance g , three Long Method instances, and several Feature Envy instances, with g a prerequisite of each envy instance by the first rule of Table I. A detection-order plan that begins by fixing Feature Envy moves methods between classes to reduce data affinity, but because g still holds the responsibilities those methods touch, the move either reintroduces coupling or is undone when g is finally decomposed. The intermediate module is structurally inconsistent, and effort is wasted on transformations that a later step invalidates. The prerequisite-respecting plan resolves g first; the extraction redistributes the responsibilities, and the set of genuine Feature Envy instances that remain is now well defined and can be resolved once, correctly. We call the first situation a cascading violation and define it operationally next.

Definition 2 (Cascading violation). *A transformation step t that resolves smell v is a cascading violation if it introduces a new smell, or invalidates a previously completed step, that would not have arisen had every prerequisite of v been resolved before t . The cascading-violation count of an execution is the number of such steps.*

We also record net progress. With R the set of smells resolved and I the set of new smells introduced across an execution,

$$NSR = |R| - |I| \quad (4)$$

the net smell resolution, is the primary effectiveness measure of a plan.

V. The SEQ-REFACTOR Algorithm

The algorithm computes an impact-forward linear extension by a single traversal that keeps, at all times, the set of smells whose prerequisites are already satisfied, and among those repeatedly selects the highest-impact one. This is Kahn's topological sort [23] in which the ready set is realised as a maximum priority queue keyed by (1), which is exactly why the procedure unifies dependency safety with impact-first selection: the priority queue supplies the best-first behaviour while the in-degree bookkeeping supplies the topological guarantee. When the module's dependencies contain a cycle, the traversal cannot consume every vertex; the residual is decomposed into strongly-connected components by Tarjan's algorithm [24], each non-trivial component is emitted as a single compound task for human review, and ordering continues on the acyclic condensation.

Algorithm 1. SEQ-REFACTOR ordering

Input: smell-dependency graph $G = (V, E)$; weights (a, b, g)
Output: ordered agenda π ; escalation set S

```

1  compute Impact(v) for all v in V                                (Eq. 1)
2  indeg(v) <- number of prerequisites of v, for all v
3  Q <- max-priority-queue of { v : indeg(v) = 0 } by Impact
4  pi <- < > ; S <- { }
5  while Q not empty do
6    v <- ExtractMax(Q)
7    append v to pi
8    foreach successor w of v do
9      indeg(w) <- indeg(w) - 1
10     if indeg(w) = 0 then insert w into Q by Impact(w)
11 if |pi| < |V| then // vertices trapped in cycles
12   R <- V \ pi
13   foreach non-trivial SCC C of G[R] by Tarjan do
14     add compound task C to S for human review
15   order condensation of G[R]; append its acyclic parts to pi
16 return (pi, S)
```

A. Correctness

Two properties hold. First, *safety*: because a vertex is inserted into Q only when its in-degree reaches zero, no smell is emitted before all of its prerequisites, so the acyclic portion of the output is a valid linear extension and respects every edge of E . Second, *impact-forward selection*: at each step the algorithm emits the maximum-impact vertex among those currently eligible, which is the greedy choice for the objective in (2). The algorithm does not guarantee the global optimum of (3), since a greedy choice can forgo a higher discounted return available only through a different eligible

sequence; we therefore treat ordering quality as an empirical question. Cycles are never resolved silently: any smells that cannot be linearly ordered are returned in S and escalated, which encodes the design decision to prefer safety over automation.

B. Complexity

Computing impact scores is linear in the elements referenced by the vertices. The priority-queue traversal performs $O(|V|)$ extractions and $O(|E|)$ decrease-key or insert operations, giving $O((|V| + |E|) \log |V|)$ with a binary heap. Tarjan's decomposition of the residual is $O(|V| + |E|)$. The overall cost is therefore $O((|V| + |E|) \log |V|)$, which is negligible relative to smell detection, transformation generation, and verification, and scales comfortably to the smell counts of large modules.

VI. Agentic Execution Architecture

The ordering of Algorithm 1 is executed by a stateful pipeline orchestrated with LangGraph, chosen because it provides deterministic, auditable, conditionally branching workflows with explicit shared state. The pipeline threads a state object that holds the module, the current smell-dependency graph, the metric baseline, and the agenda, through the following stages.

Detect and build. A static analyser reports the smells of the module, and the graph builder constructs G from catalogue rules and structural co-location, recording edge provenance.

Order. Algorithm 1 produces the agenda π and the escalation set S . Compound tasks in S are routed to human review before any automated step proceeds on their elements.

Retrieve and generate. For the next smell in π , a retrieval layer built with LangChain and a document index supplies context to the generator. Vector retrieval returns semantically similar code, and structural retrieval traverses a code-property graph of call, dependency, and inheritance edges to supply the dependency context whose absence is the documented cause of hallucinated, architecturally blind transformations [18]. Grounding here serves the generator; it is a component, not the contribution.

Verify and gate. The candidate transformation is applied in an isolated worktree and evaluated on three signals: the five-family metric delta of Section II-D, execution of the module's test suite, and an architectural constraint check on interface surface and dependency direction. If the evidence is acceptable the change is committed; otherwise it is rolled back, logged, and, if it repeatedly fails, escalated.

Re-detect. After an accepted step the analyser runs again on the changed module, and the graph is updated: resolving one smell removes it and any edges it anchored, and may reveal genuinely new vertices. This closing of the loop is what keeps the agenda correct as structure changes, and it directly supplies the state tracking whose absence

RefactorBench identifies as the dominant agent failure [18]. The generator is never asked to remember what has changed; the pipeline maintains that state and re-plans from it.

VII. Experimental Design

A. Research Questions and Hypotheses

- **RQ1.** Does dependency-aware ordering reduce cascading violations relative to unordered, impact-only, and topology-only execution?
- **RQ2.** Does dependency-aware ordering improve net smell resolution (Eq. 4)?
- **RQ3.** How frequently do modules contain smell-dependency cycles, and does strongly-connected-component escalation prevent unsafe automated transformation of those cases?
- **RQ4.** How sensitive are cascading violations and net smell resolution to the impact weights (α , β , γ) and the discount δ ?
- **H1.** SEQ-REFACTOR ordering yields significantly fewer cascading violations than unordered execution.
- **H2.** SEQ-REFACTOR ordering yields higher net smell resolution than both unordered and topology-only execution.
- **H3.** Impact-forward placement yields higher early cumulative quality gain, measured as the area under the quality-versus-step curve, than topology-only ordering with arbitrary priority.

B. Variables

The primary independent variable is the ordering strategy, with four levels: SEQ-REFACTOR (impact-forward and dependency-safe), impact-only (best-first by (1) with the topological constraint removed), topology-only (a valid linear extension with arbitrary priority), and unordered (smells addressed in detection order). To separate the effect of ordering from the effect of the generator, the generator is a controlled factor with two levels: an LLM provider and an in-house deterministic Extract Method baseline. Dependent variables are the cascading-violation count, net smell resolution, ordering validity (the fraction of steps whose prerequisites were satisfied when executed), escalation rate, and the cumulative quality-delta trajectory that underlies H3.

C. Subjects

Two tiers of subjects are used. The controlled synthetic tier consists of Java modules of twenty to forty-five files into which smells are injected together with a known

dependency structure, so that the correct order, the expected post-state, and the specific cascading pitfalls are all known exactly and both false rejection and false acceptance are measurable. The open-source tier draws submodules from widely studied Java projects; developer-executed refactoring commits recovered with a refactoring miner [13] provide a reference order against which automated plans are compared. The instrument is Java-only in this first increment for the reasons given in the scope.

D. Protocol and Analysis

Each strategy is run on each subject under each generator. After every transformation step the analyser re-runs and the metric families are recomputed, so that new smells are attributed to steps and cascading violations are counted as they occur. Results are aggregated across subjects with non-parametric tests appropriate to paired designs, and effect sizes are reported alongside significance, following standard guidance for empirical software engineering experiments [25]. Every run is reproducible: generation is seeded, raw model outputs are recorded rather than re-queried, and per-step artefacts (the graph, the agenda, the metric deltas, and the verdicts) are persisted so that any reported number can be re-derived.

VIII. Preliminary Pilot

To establish feasibility of the pipeline and to exhibit the failure mode that ordering prevents, we ran an illustrative pilot on a single controlled synthetic Java module. A static analyser detected eight smell instances, from which the smell-dependency graph was constructed and Algorithm 1 produced the execution order

God Class → *Long Method* (×3) → *Message Chains* (×2) → *Big Switch*.

Resolving the God Class first redistributed responsibilities and thereby fixed which method boundaries the subsequent Long Method extractions should follow; when the same eight smells were instead addressed in detection order, an early method extraction was performed against boundaries that the later God Class decomposition changed, producing an intermediate inconsistency of the kind captured by our cascading-violation definition. The dependency-aware order avoided that step. Consistent with our commitment to report only what was measured, we stress that this is a single directional case: with eight instances it is illustrative of the mechanism, not evidence of an effect size, and it is insufficient for statistical inference. The controlled study of Section VII is designed to provide that inference across many subjects, and the pilot's role is limited to demonstrating that the pipeline runs end to end and that the predicted failure mode is real and observable.

IX. Discussion

A. Contributions in Perspective

The theoretical contribution is the formalisation of smell-dependency ordering as an impact-forward linear extension of a prerequisite partial order, with a precise notion of cascading violation. The methodological contribution is a controlled evaluation with defined baselines, ground-truth subjects, and dependent measures that isolate ordering from generation. The technical contribution is a stateful pipeline that unifies impact-prioritised topological ordering with cycle escalation and re-planning. The practical contribution is safer automated refactoring of modules that carry many interacting smells, and a reusable ordering layer that any transformation tool can adopt.

B. Relationship to Model Refactoring

The ordering formalism is agnostic to the artefact being refactored. Its vertices are smells and its edges are structural prerequisites, neither of which is specific to source code. The same discipline applies to model-level smells in UML and domain models, where responsibility misalignment and cyclic dependency accumulate as they do in code, connecting this work to the model-refactoring line of the supervising group [26], [27], [28]. Extending SEQ-REFACTOR to order model transformations, and to propagate an ordering across the requirements-to-code artefact hierarchy, is a natural continuation.

C. Threats to Validity

Construct validity. The edge set depends on catalogue rules whose correctness is a modelling choice. We mitigate this by grounding the rules in the refactoring literature, by supplementing them with structural co-location derived from the module itself, by recording edge provenance for audit, and by the weight-sensitivity analysis of RQ4. A second construct concern is the novelty framing itself: because search-based refactoring scheduling is an established field, the contribution rests on the specific conjunction of a safety-framed formalisation, dependency-correct ordering, and an agentic pipeline rather than on ordering per se, and a fair evaluation should therefore, where feasible, add a scheduling-style baseline that optimises for quality or effort and report where our safety objective and its results diverge from such a schedule. *Internal*

validity. LLM non-determinism could confound results; we control for it by seeding and recording generation and by including the deterministic rule baseline as a generator level, so that ordering effects are visible independently of the generator. *External validity.* The instrument is Java-only and the controlled tier is synthetic; the open-source tier and reference orders from mined refactorings mitigate ecological concerns, and generalisation to dynamically typed languages is explicit future work. *Conclusion validity.* The pilot is a single case; the powered study of Section VII, with non-parametric tests and effect sizes, is designed to support statistical conclusions.

D. Limitations

The present increment targets Java, derives edges from a curated catalogue rather than a learned model, and escalates rather than resolves dependency cycles. The last of these is a deliberate safety choice: within a genuine dependency cycle no linear order can respect all constraints, and presenting such a case to a human is safer than allowing the pipeline to break the cycle arbitrarily.

X. Conclusion

Multi-smell refactoring is not a set of independent single-smell tasks, and the order in which interacting smells are resolved is a safety property of the plan. SEQ-REFACTOR contributes a formal, graph-theoretic treatment of that property, framed as a behavioural-safety concern that complements existing search-based refactoring scheduling [29]: a smell-dependency graph with audited prerequisite edges, an impact score over coupling, complexity, and co-occurrence, and an algorithm that computes an impact-forward, dependency-safe order in near-linear time while surfacing dependency cycles for human review. Embedded in a stateful LangGraph pipeline that grounds generation with structural retrieval, gates transformations on metric and test verification, and re-plans after every accepted step, the ordering targets exactly the stateful multi-file reasoning where autonomous refactoring has been shown to fail. We have specified a controlled evaluation to quantify its effect and reported a directional pilot that exhibits the cascading failure it prevents. The ordering layer is artefact-agnostic and reusable, and its extension to model-level and requirements-level refactoring, and to dynamically typed languages, defines the next stage of this research.

Acknowledgment

The author thanks

.....

References

- [1] T. J. McCabe, "A complexity measure," IEEE Trans. Softw. Eng., vol. SE-2, no. 4, pp. 308-320, Dec. 1976, doi: 10.1109/TSE.1976.233837. <https://doi.org/10.1109/TSE.1976.233837>
- [2] S. R. Chidamber and C. F. Kemerer, "A metrics suite for object oriented design," IEEE Trans. Softw. Eng., vol. 20, no. 6, pp. 476-493, Jun. 1994, doi: 10.1109/32.295895. <https://doi.org/10.1109/32.295895>
- [3] D. Coleman, D. Ash, B. Lowther, and P. Oman, "Using metrics to evaluate software system maintainability," Computer, vol. 27, no. 8, pp. 44-49, Aug. 1994, doi: 10.1109/2.303623. <https://doi.org/10.1109/2.303623>
- [4] M. Fowler, Refactoring: Improving the Design of Existing Code, 2nd ed. Boston, MA, USA: Addison-Wesley, 2018. <https://martinfowler.com/books/refactoring.html>
- [5] W. F. Opdyke, "Refactoring object-oriented frameworks," Ph.D. dissertation, Dept. Comput. Sci., Univ. of Illinois at Urbana-Champaign, Urbana, IL, USA, 1992. <https://www.laputan.org/pub/papers/opdyke-thesis.pdf>
- [6] R. P. L. Buse and W. R. Weimer, "Learning a metric for code readability," IEEE Trans. Softw. Eng., vol. 36, no. 4, pp. 546-558, Jul./Aug. 2010, doi: 10.1109/TSE.2009.70. <https://doi.org/10.1109/TSE.2009.70>
- [7] S. Scalabrino, M. Linares-Vasquez, R. Oliveto, and D. Poshyvanyk, "A comprehensive model for code readability," IEEE Trans. Softw. Eng., vol. 47, no. 3, pp. 595-613, Mar. 2021, doi: 10.1109/TSE.2019.2901468. <https://doi.org/10.1109/TSE.2019.2901468>
- [8] G. A. Campbell, "Cognitive complexity: An overview and evaluation," in Proc. Int. Conf. Tech. Debt (TechDebt), 2018, pp. 57-58, doi: 10.1145/3194164.3194186. <https://doi.org/10.1145/3194164.3194186>
- [9] R. C. Martin, Agile Software Development, Principles, Patterns, and Practices. Upper Saddle River, NJ, USA: Prentice Hall, 2003. <https://www.pearson.com/en-us/subject-catalog/p/agile-software-development-principles-patterns-and-practices/P200000003337>
- [10] J.-L. Letouzey, "The SQALE method for evaluating technical debt," in Proc. 3rd Int. Workshop Managing Tech. Debt (MTD), 2012, pp. 31-36, doi: 10.1109/MTD.2012.6225997. <https://doi.org/10.1109/MTD.2012.6225997>
- [11] A. Tornhill and M. Borg, "Code red: The business impact of code quality - a quantitative study of 39 proprietary production codebases," in Proc. Int. Conf. Tech. Debt (TechDebt), 2022, pp. 11-20, doi: 10.1145/3524843.3528091. <https://doi.org/10.1145/3524843.3528091>
- [12] N. Tsantalis, T. Chaikalis, and A. Chatzigeorgiou, "Ten years of JDeodorant: Lessons learned from the hunt for smells," in Proc. IEEE 25th Int. Conf. Softw. Anal., Evol. and Reengineering (SANER), 2018, pp. 4-14, doi: 10.1109/SANER.2018.8330192. <https://doi.org/10.1109/SANER.2018.8330192>
- [13] N. Tsantalis, A. Ketkar, and D. Dig, "RefactoringMiner 2.0," IEEE Trans. Softw. Eng., vol. 48, no. 3, pp. 930-950, Mar. 2022, doi: 10.1109/TSE.2020.3007722. <https://doi.org/10.1109/TSE.2020.3007722>
- [14] D. Pomian, A. Bellur, M. Dilhara, Z. Kurbatova, E. Bogomolov, A. Sokolov, T. Bryksin, and D. Dig, "EM-Assist: Safe automated ExtractMethod refactoring with LLMs," in Proc. 32nd ACM Int. Conf. Found. Softw. Eng. (FSE Companion), 2024, pp. 582-586, doi: 10.1145/3663529.3663803. <https://doi.org/10.1145/3663529.3663803>
- [15] D. Wu, F. Mu, L. Shi, Z. Guo, K. Liu, W. Zhuang, Y. Zhong, and L. Zhang, "iSMELL: Assembling LLMs with expert toolsets for code smell detection and refactoring," in Proc. 39th IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE), 2024, pp. 1345-1357, doi: 10.1145/3691620.3695508. <https://doi.org/10.1145/3691620.3695508>
- [16] Y. Xu, F. Lin, J. Yang, T. Chen, and N. Tsantalis, "MANTRA: Enhancing automated method-level refactoring with contextual RAG and multi-agent LLM collaboration," arXiv:2503.14340, 2025. <https://arxiv.org/abs/2503.14340>

- [17] K. Oueslati, M. Lamothe, and F. Khomh, "RefAgent: A multi-agent LLM-based framework for automatic software refactoring," arXiv:2511.03153, 2025. <https://arxiv.org/abs/2511.03153>
- [18] D.D. Gautam, S. Garg, J. Jang, N. Sundaresan, and R. Z. Moghaddam, "RefactorBench: Evaluating stateful reasoning in language agents through code," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2025, arXiv:2503.07832. <https://arxiv.org/abs/2503.07832>
- [19] H. Zhang et al., "SmellBench: Towards fine-grained evaluation of code agents on refactoring tasks," arXiv:2606.05574, 2026. <https://arxiv.org/abs/2606.05574>
- [20] S. A. Vidal, C. Marcos, and J. A. Diaz-Pace, "An approach to prioritize code smells for refactoring," *Automated Softw. Eng.*, vol. 23, no. 3, pp. 501-532, 2016, doi: 10.1007/s10515-014-0175-x. <https://doi.org/10.1007/s10515-014-0175-x>
- [21] A. Yamashita and L. Moonen, "Exploring the impact of inter-smell relations on software maintainability: An empirical study," in *Proc. 35th Int. Conf. Softw. Eng. (ICSE)*, 2013, pp. 682-691, doi: 10.1109/ICSE.2013.6606614. <https://doi.org/10.1109/ICSE.2013.6606614>
- [22] F. Palomba, G. Bavota, M. Di Penta, R. Oliveto, D. Poshyvanyk, and A. De Lucia, "Mining version histories for detecting code smells," *IEEE Trans. Softw. Eng.*, vol. 41, no. 5, pp. 462-489, May 2015, doi: 10.1109/TSE.2014.2372760. <https://doi.org/10.1109/TSE.2014.2372760>
- [23] A. B. Kahn, "Topological sorting of large networks," *Commun. ACM*, vol. 5, no. 11, pp. 558-562, Nov. 1962, doi: 10.1145/368996.369025. <https://doi.org/10.1145/368996.369025>
- [24] R.E. Tarjan, "Depth-first search and linear graph algorithms," *SIAM J. Comput.*, vol. 1, no. 2, pp. 146-160, Jun. 1972, doi: 10.1137/0201010. <https://www.cs.cmu.edu/~cdm/resources/Tarjan1972-sccs.pdf>
- [25] C. Wohlin, P. Runeson, M. Host, M. C. Ohlsson, B. Regnell, and A. Wesslen, *Experimentation in Software Engineering*. Berlin, Germany: Springer, 2012, doi: 10.1007/978-3-642-29044-2. <https://doi.org/10.1007/978-3-642-29044-2>
- [26] M. Štolc, I. Polášek, "A visual based framework for the model refactoring techniques," in *Proc. IEEE 8th Int. Symp. Applied Machine Intelligence and Informatics (SAMi)*, Herl'any, Slovakia, 2010, pp. 77-82, doi: 10.1109/SAMI.2010.5423766. <https://doi.org/10.1109/SAMI.2010.5423766>
- [27] I. Polášek, S. Snopko, and I. Kapustik, "Automatic identification of the anti-patterns using the rule-based approach," in *Proc. IEEE 10th Jubilee Int. Symp. Intelligent Systems and Informatics (SISY)*, Subotica, Serbia, 2012, pp. 283-286, doi: 10.1109/SISY.2012.6339530. <https://doi.org/10.1109/SISY.2012.6339530>
- [28] I. Polášek, P. Liska, J. Kelemen, and J. Lang, "On extended similarity scoring and bit-vector algorithms for design smell detection," in *Proc. IEEE 16th Int. Conf. Intelligent Engineering Systems (INES)*, Lisbon, Portugal, 2012, pp. 115-120, doi: 10.1109/INES.2012.6249814. <https://doi.org/10.1109/INES.2012.6249814>
- [29] A. Ouni, M. Kessentini, H. Sahraoui, and M. Boukadoum, "Maintainability defects detection and correction: A multi-objective approach," *Automated Softw. Eng.*, vol. 20, no. 1, pp. 47-79, 2013, doi: 10.1007/s10515-011-0098-8. <https://doi.org/10.1007/s10515-011-0098-8>
- [30] H. Liu, Z. Ma, W. Shao, and Z. Niu, "Schedule of bad smell detection and resolution: A new way to save effort," *IEEE Trans. Softw. Eng.*, vol. 38, no. 1, pp. 220-235, Jan./Feb. 2012, doi: 10.1109/TSE.2011.9. <https://doi.org/10.1109/TSE.2011.9>
- [31] F. Palomba, G. Bavota, M. Di Penta, F. Fasano, R. Oliveto, and A. De Lucia, "A large-scale empirical study on the lifecycle of code smell co-occurrences," *Inf. Softw. Technol.*, vol. 99, pp. 1-10, 2018, doi: 10.1016/j.infsof.2018.02.004. <https://doi.org/10.1016/j.infsof.2018.02.004>